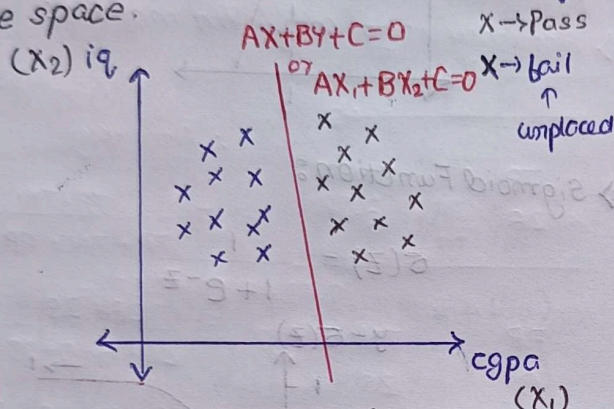


Logistic Regression

Perceptron Trick

⇒ Prerequisite of Logistic Regression :

① Basic LR assumes a linear relationship b/w the features and the log-odds, resulting in a linear decision boundary in the feature space.



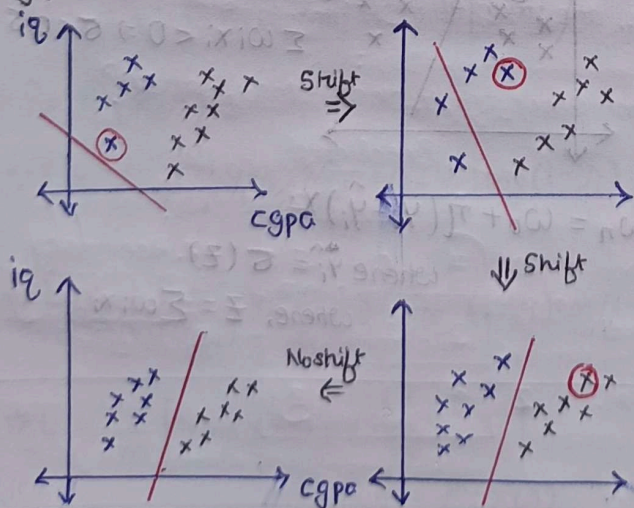
② Target should be categorical.

③ Observations should generally be independent.

④ No severe multicollinearity.

⇒ What's the idea of Perceptron? Look at a training point, check whether the model classified it correctly, and if it's wrong, adjust the weights so that the decision boundary moves in the correct direction.

eg.,



⇒ But, how to label region?

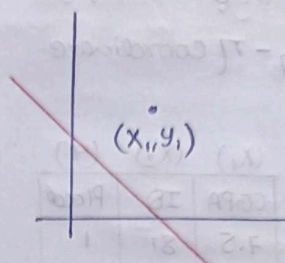
$$AX + BY + C = 0$$

If X_1, y_1 then

$$AX_1 + BY_1 + C = 0 \rightarrow \text{on line}$$

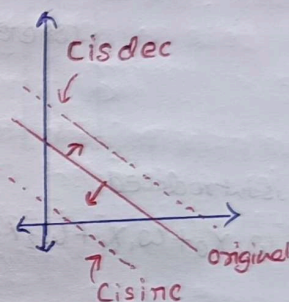
$$AX_1 + BY_1 + C > 0 \rightarrow \text{above line (ve)}$$

$$AX_1 + BY_1 + C < 0 \rightarrow \text{below line (-ve)}$$

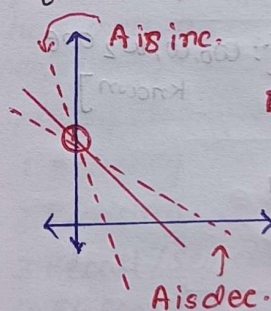


⇒ Btw, how the transformation is done? Transformation is done by making changes in A, B and C.

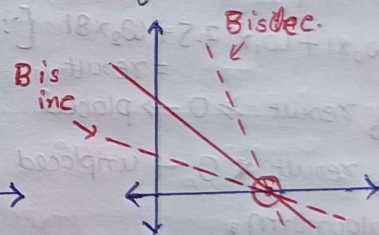
(i) If C is changed:



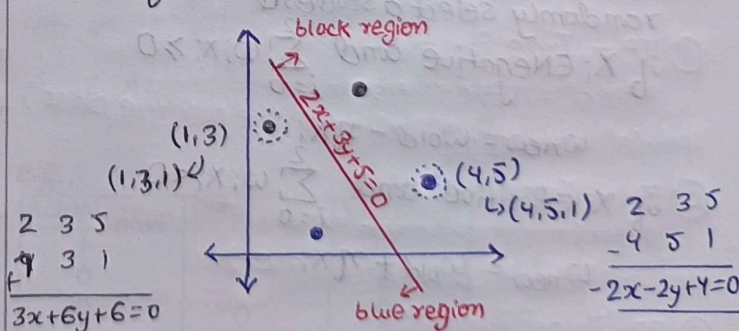
(ii) If A is changed:



(iii) If B is changed:



⇒ Transformation basic example:



∴ Learning rate is used to make sure that transformation is done slowly,

$$(coeff)_{new} = (coeff)_{old} - \eta \text{ coordinate}$$

⇒ Algorithm:

algorithm:

CGPA	IS	Placed
7.5	81	1
8.9	109	1
7.0	81	0

$\Rightarrow$

X_0	(X_1)	(X_2)	(Y)
CGPA	IS	Placed	
7.5	81	1	
8.9	109	1	
7.0	81	0	

$$\therefore AX + BY + C = 0$$

$$\Rightarrow \omega_0 + \omega_1 X_1 + \omega_2 X_2 = 0$$

where, $\omega_0 = C$

$\omega_1 = A$

$\omega_2 = B$

∴ X_0 is introduced,

$$\omega_0 X_0 + \omega_1 X_1 + \omega_2 X_2 = 0 \Rightarrow \sum_{i=0}^2 \omega_i X_i = 0$$

∴ to make prediction for 1st student,

$$\omega_0 \times 1 + \omega_1 \times 7.5 + \omega_2 \times 81 \quad [\because \omega_0, \omega_1, \omega_2 \text{ are known}]$$

if result $\geq 0 \rightarrow$ placed

if result $\leq 0 \rightarrow$ unplaced

∴ the algorithm,

$$\text{epoch} = 1000, \eta = 0.01$$

for i in range(epoch):

randomly select a student

if $X_i \in \text{Negative}$ and $\sum_{i=0}^2 \omega_i X_i \geq 0$

$$\omega_{new} = \omega_{old} - \eta X_i$$

if $X_i \in \text{Positive}$ and $\sum_{i=0}^2 \omega_i X_i < 0$

$$\omega_{new} = \omega_{old} + \eta X_i$$

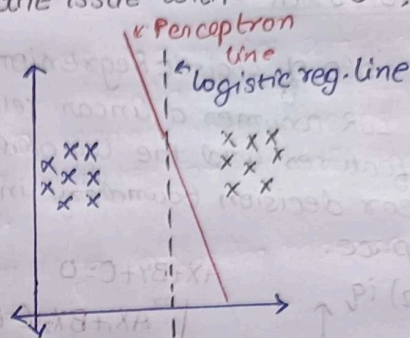
OR, optimised be:

For i in epochs:

random student

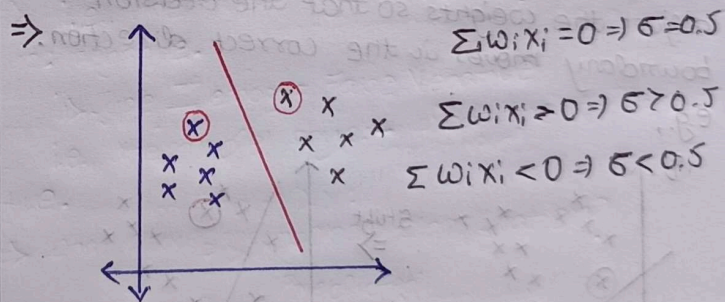
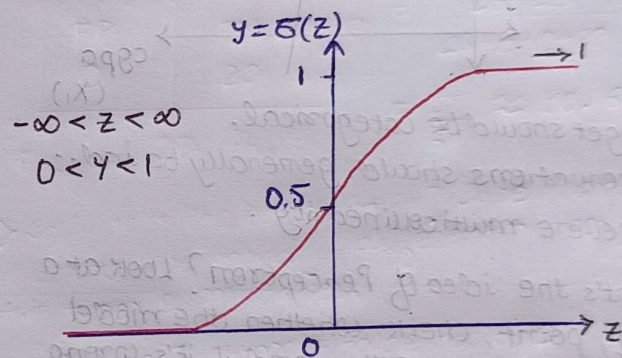
$$\omega_n = \omega_0 + \eta (y_i - \hat{y}_i) X_i$$

⇒ What's the issue with this algo?



⇒ Sigmoid Function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



$$\omega_n = \omega_0 + \eta (y_i - \hat{y}_i) X_i$$

where $\hat{y}_i = \sigma(z)$

where, $z = \sum \omega_i X_i$

⇒ Loss function / Binary cross entropy:

$$L = -\frac{1}{n} \sum_{i=1}^n y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)$$

⇒ Also,

$$\sigma'(x) = \sigma(x)[1 - \sigma(x)]$$

Gradient Descent in LOR: (Rows = m; Columns = n)

	1	2	3	...	n	y	$\hat{y}$
1	x_{11}	x_{12}	x_{13}	...	x_{1n}	y_1	$\hat{y}_1$
2	x_{21}	x_{22}	x_{23}	...	x_{2n}	y_2	$\hat{y}_2$
...	...	...	...	...	...	...	...
m	x_{m1}	x_{m2}	x_{m3}	...	x_{mn}	y_m	$\hat{y}_m$

$$\therefore \hat{y}_1 = \sigma(\omega_0 + \omega_1 x_{11} + \omega_2 x_{12} + \dots + \omega_n x_{1n})$$

$$\hat{y}_2 = \sigma(\omega_0 + \omega_1 x_{21} + \omega_2 x_{22} + \dots + \omega_n x_{2n})$$

$$\therefore \hat{y} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_m \end{bmatrix} = \begin{bmatrix} \sigma(\omega_0 + \omega_1 x_{11} + \omega_2 x_{12} + \dots + \omega_n x_{1n}) \\ \sigma(\omega_0 + \omega_1 x_{21} + \omega_2 x_{22} + \dots + \omega_n x_{2n}) \\ \vdots \\ \sigma(\omega_0 + \omega_1 x_{m1} + \omega_2 x_{m2} + \dots + \omega_n x_{mn}) \end{bmatrix}$$

$$\Rightarrow \hat{y} = \sigma \left(\begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1n} \\ 1 & x_{21} & x_{22} & \dots & x_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m1} & x_{m2} & \dots & x_{mn} \end{bmatrix} \begin{bmatrix} \omega_0 \\ \omega_1 \\ \vdots \\ \omega_n \end{bmatrix} \right)$$

$$\Rightarrow \hat{y} = \sigma(X\omega)$$

$$\therefore L = -\frac{1}{m} \sum_{i=1}^m y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)$$

$$\Rightarrow L = -\frac{1}{m} \left[\sum_{i=1}^m y_i \log(\hat{y}_i) + \sum_{i=1}^m (1-y_i) \log(1-\hat{y}_i) \right]$$

$$\Rightarrow L = -\frac{1}{m} [y \log \hat{y} + (1-y) \log(1-\hat{y})]$$

$$\text{where } \hat{y} = \sigma(X\omega)$$

Now,

$$\frac{dL}{d\omega} = -\frac{1}{m} (y - \hat{y})X$$

We know,

$$\omega = \omega - \eta \frac{\Delta L}{\Delta \omega}$$

$$\Rightarrow \omega = \omega + \eta \frac{1}{m} (y - \hat{y})X$$

where,

$$\omega = \begin{bmatrix} \omega_0 \\ \omega_1 \\ \vdots \\ \omega_n \end{bmatrix}_{(n+1) \times 1} \quad X = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1n} \\ 1 & x_{21} & x_{22} & \dots & x_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m1} & x_{m2} & \dots & x_{mn} \end{bmatrix}_{(m, n+1)}$$

$$y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}_{m \times 1} \quad \hat{y} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_m \end{bmatrix}_{m \times 1}$$

Classification Metrics:

⇒ These are measures used to evaluate how well a classification model is performing.

⇒ Main classification metrics are:

1. Accuracy
2. Precision
3. Recall / Sensitivity
4. F1-Score
5. ROC-AUC etc.

⇒ Accuracy: It's the portion of total prediction that the model classified correctly.

$$\therefore \text{Accuracy} = \frac{\text{No. of correct Predictions}}{\text{Total no. of predictions}}$$

e.g.,

Actual Label	LoR Prediction	Decision Tree Prediction
1	1	1
0	0	0
0	0	0
0	0	0
0	0	0
0	0	0
0	0	0
0	0	0
0	0	0
0	0	0

$$\therefore \text{LoR accuracy} = \frac{8}{10} = 80\%$$

$$\text{Decision Tree accuracy} = \frac{9}{10} = 90\%$$

⇒ How much Accuracy is good? There is no universal "good accuracy" value, it depends on:

- The problem
- The dataset
- The cost of mistakes
- The application/domain

e.g.,
Accuracy of 90% for food order prediction is good, but accuracy of 90% for disease prediction system is not good.

⇒ Major problem with accuracy: It treats all classification mistakes equally and doesn't tell us what type of mistake the model is making.

⇒ What's the solution then? Confusion matrix. It gives a detailed breakdown of the actual vs predicted classes.

Prediction

	1	0
Actual 1	True Positive (TP)	False Negative (FN)
Actual 0	False Positive (FP)	True Negative (TN)

⇒ What does each cell mean?

① TP - True Positive

Actual = 1, Prediction = 1
(Model correctly predicted the positive class)

② FN - False Negative

Actual = 1, Prediction = 0
(Model missed a positive case)

③ FP - False Positive

Actual = 0, Prediction = 1
(Model incorrectly predicted a positive case)

④ TN - True Negative

Actual = 0, Prediction = 0

(Model correctly predicted the negative class)

⇒ Why it is called "Confusion" matrix? Because it shows where the model gets confused. e.g.,

FN = 6 means the model confused actual positive cases with negative predictions. And, FP = 0 means there were no cases where the model confused an actual negative with a positive.

⇒ Accuracy using confusion matrix:

TP 26	FN 6
FP 0	TN 29

∴ Accuracy

$$= \frac{TP + TN}{TP + TN + FP + FN}$$

$$= \frac{26 + 29}{26 + 6 + 0 + 29}$$

$$= \frac{55}{61} \approx 90.16\%$$

⇒ So, what's the error in it? Type I and Type II errors are basically different names for the two types of mistakes.

i) Type I error = False Positive

i.e., the model says: +ve; but the truth is: negative.

ii) Type II error = False Negative

i.e., the model says: +ve; but the truth is: -ve.

⇒ What about multi-class classification?

	Predicted		
	0	1	2
Actual 0	7	0	5
Actual 1	2	21	6
Actual 2	9	1	13

∴ Accuracy

$$= \frac{\text{sum of diag.}}{\text{Total}}$$

⇒ when can Accuracy be misleading? It can be misleading when the dataset is imbalanced.
 e.g., Suppose an airport has 10,000 passengers out of them 9,990 are normal and 10 are terrorists.

∴ Normal : terrorist = 9990 : 10

So, if a model be:

	Predicted		
	1	0	
Terrorist(1)	TP = 0	FN = 10	Actual
	FP = 0	TN = 9990	
Normal(0)			

$$\therefore \text{accuracy} = \frac{0 + 9990}{0 + 9990 + 0 + 10} \approx 99.9\%$$

∴ it detected 0 out of 10 terrorists and it allowed 10/10 terrorists.

∴ accuracy can be misleading when minority class is the important class.

⇒ So we have another metrics called Precision.

⇒ What is Precision? It tells us, out of all the observations that the model predicted as positive, how many are actually positive.

e.g., If 100 emails are predicted as spam, out of which 80 are actually spam. Then,

$$\text{Precision} = \frac{80}{100} \approx 80\%$$

Also, in terms of confusion matrix:

	Predicted		
	1	0	
Actual	TP = 80	FN = 20	∴ Precision
	FP = 20	TN = 80	
	= $\frac{TP}{TP + FP}$		

⇒ If two models have same accuracy then which one do we choose? We look for higher precision one is better.

Model A:

TP = 80	FN = 20
FP = 20	TN = 80

Model B:

TP = 90	FN = 10
FP = 10	TN = 90

clearly, accuracy is same for both, so we will look for precision.

$$\therefore \text{Precision}_A = \frac{80}{80 + 20} = 80\%$$

$$\text{Precision}_B = \frac{90}{90 + 10} = 90\%$$

$$\therefore \text{Precision}_B > \text{Precision}_A$$

∴ model B is better than A, if avoiding false positives is important.

⇒ What is Recall? It tells us, out of all the actual Positive cases, how many the model correctly identified as Positive. It's also called sensitivity.

TP = 90	FN = 10
FP = 10	TN = 90

$$\therefore \text{Recall} = \frac{TP}{TP + FN}$$

e.g., If 100 patients actually have a disease, and the model detects TP=90 and FN=10, then,

$$\text{Recall} = \frac{90}{90 + 10} = 90\%$$

∴ the model was able to catch 90% of all actual disease cases.

⇒ If two models have the same accuracy, which one should we choose? We should think of what kind of error is more dangerous for our problem.

∴ Type I error = FP

Type II error = FN

Also, Precision is concerned with FP

Recall is concerned with FN

∴ (i) If Type I error is more dangerous then we will prefer higher precision,

(ii) If Type II error is more dangerous then we will prefer higher Recall.

⇒ What if we don't know which is more important? FP or FN? We use F1 score.

⇒ What is F1 score? It is a classification metric that combines Precision and Recall into a single score using their harmonic mean.

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$= \frac{2PR}{P+R}$$

⇒ Multi-Class Precision and Recall:

		Predicted			
		Dog	Cat	Rabbit	Total
Actual	Dog	25	5	10	40
	Cat	0	30	4	34
	Rabbit	4	10	20	34
	Total	29	45	34	

Predicted total

$$P_{\text{Dog}} = \frac{25}{29} = 0.86, P_{\text{Cat}} = \frac{30}{45} = 0.66,$$

$$P_{\text{Rabbit}} = \frac{20}{34} = 0.58$$

$$\therefore \text{Macro Precision} = \frac{0.86 + 0.66 + 0.58}{3} = 0.70$$

$$\therefore \text{Weighted Precision} = \frac{40}{108} \times 0.86 + \frac{34}{108} \times 0.66 + \frac{34}{108} \times 0.58 = 0.71$$

$$R_{\text{Dog}} = \frac{25}{40} = 0.62, R_{\text{Cat}} = \frac{30}{34} = 0.88,$$

$$R_{\text{Rabbit}} = \frac{20}{34} = 0.58$$

$$\therefore \text{Macro Recall} = \frac{0.62 + 0.88 + 0.58}{3} = 0.69$$

ly,

$$F1_{\text{Dog}} = \frac{2P_{\text{Dog}}R_{\text{Dog}}}{P_{\text{Dog}} + R_{\text{Dog}}}$$

ROC Curve?

⇒ Why do we need ROC curve? We know models like logistic Regression doesn't directly produce just 0 or 1. It produces a probability. To convert probabilities into classes, we choose a threshold. e.g., with threshold 0.5:

$$P \geq 0.5 \rightarrow \text{Positive}$$

$$P < 0.5 \rightarrow \text{Negative}$$

But why must we always use 0.5?

We should use:

0.3, 0.4, 0.5, 0.6, 0.7, ... and changing

the threshold changes the no. of TP, FP, TN and FN.

∴ ROC helps us see the model's behavior across diff. thresholds rather than evaluating it at just one threshold.

⇒ what does ROC plot? ROC stands for Receiver Operating Characteristic. It plots TPR vs FPR.

⇒ what is TPR? True Positive Rate.

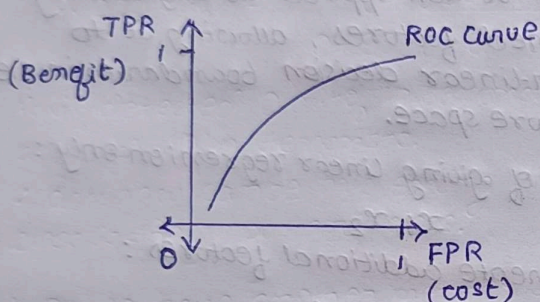
$$TPR = \frac{TP}{TP+FN} = \text{Recall} \rightarrow \text{y-axis}$$

what is FPR? False Positive Rate.

$$FPR = \frac{FP}{FP+TN} \rightarrow \text{x-axis}$$

⇒ TPR need to be maximised (i.e., FN need to be minimised). Also, FPR need to be minimised.

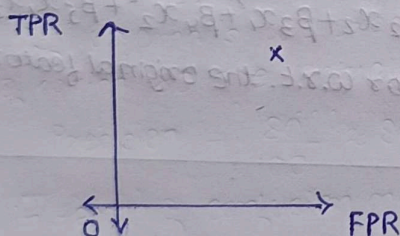
⇒ How does it look like?



⇒ Different cases:

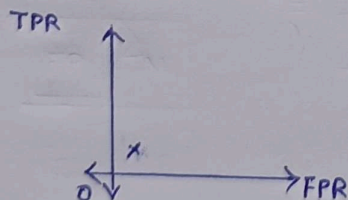
Case 1: threshold is very small

∴ FPR ↑ TPR ↑

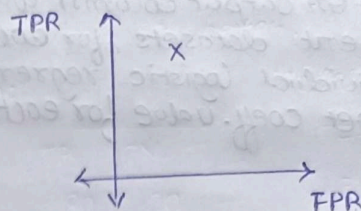


Case 2: threshold is very large

∴ FPR ↓ TPR ↓

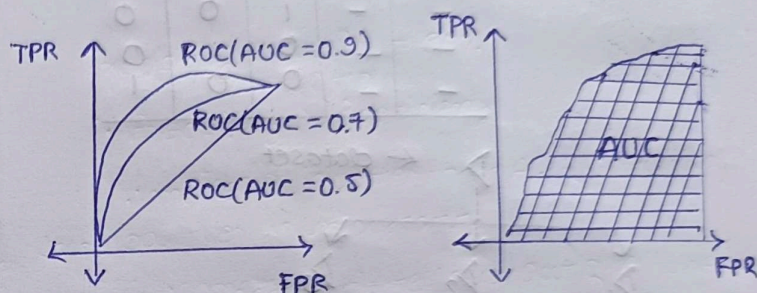


Case 3: ideal threshold



⇒ Then what is AUC? It means Area under the ROC curve. The more the curve bends towards the top-left corner, the larger the area under it.

∴ Higher AUC ⇒ better ability to distinguish classes.



⇒ AUC can be interpreted as: The probability that the model ranks a randomly chosen positive example higher than a randomly chosen negative example.

Softmax Regression:

⇒ Also, called multinomial logistic regression.

⇒ what is softmax function?

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

e.g., if Yes → 1

No → 0

Output → 3

where,

K = no. of classes

$$\sigma(z)_1 = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}}, \sigma(z)_2 = \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

$$\sigma(z)_3 = \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

⇒ How training is done for this? we use one-hot encoding on output column and later we create different datasets, for each we will create individual logistic regression model, we will get coeff. value for each model.
e.g.,

cgpa	iq	Placed?
-	-	0
-	-	1
-	-	2

Transform using OHE

cgpa	iq	=0	=1	=2
-	-	1	0	0
-	-	0	1	0
-	-	0	0	1

← dataset

↓
D₁ D₂ D₃

cgpa/iq=0 cgpa/iq=1 cgpa/iq=2

→ m_1

→ m_2

→ m_3

$\omega_1^{(0)}, \omega_2^{(0)}, \omega_0^{(0)}$ $\omega_1^{(1)}, \omega_2^{(1)}, \omega_0^{(1)}$ $\omega_1^{(2)}, \omega_2^{(2)}, \omega_0^{(2)}$

⇒ So, how prediction is done? Let us say we get a sample whose value is, $S_x = \{7, 70\}$, then for each model we will calculate the value of Z and σ .

e.g.,

m_1 yes

m_2 No

m_3 opt out

$\omega_1^{(1)}, \omega_2^{(1)}, \omega_0^{(1)}$

$\omega_1^{(2)}, \omega_2^{(2)}, \omega_0^{(2)}$

$\omega_1^{(3)}, \omega_2^{(3)}, \omega_0^{(3)}$

$$Z_1 = 7 \times \omega_1^{(1)} + 70 \times \omega_2^{(1)} + \omega_0^{(1)}$$

$$Z_2 = 7 \times \omega_1^{(2)} + 70 \times \omega_2^{(2)} + \omega_0^{(2)}$$

$$Z_3 = 7 \times \omega_1^{(3)} + 70 \times \omega_2^{(3)} + \omega_0^{(3)}$$

$$\therefore \sigma(\text{yes}) = \frac{e^{Z_1}}{e^{Z_1} + e^{Z_2} + e^{Z_3}} = 0.4 \text{ (say)}$$

$$\sigma(\text{opt}) = \frac{e^{Z_3}}{e^{Z_1} + e^{Z_2} + e^{Z_3}} = 0.25$$

$$= 0.35$$

⇒ what be the loss function here?

$$L = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log(\hat{y}_k^{(i)})$$

Polynomial Features in Log. Regression

⇒ Poly. LoR is LoR applied to polynomially transformed features, allowing it to learn non-linear decision boundaries in the original feature space.

∴ instead of giving linear regression only:

x_1, x_2

we can create additional features:

$x_1^2, x_2^2, x_1 x_2$

∴ the model becomes:

$$Z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_1^2 + \beta_4 x_2^2 + \beta_5 x_1 x_2$$

and the decision boundary is:

$$\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_1^2 + \beta_4 x_2^2 + \beta_5 x_1 x_2 = 0$$

This is non-linear w.r.t. the original features.